LAB ASSIGNMENT 20.3

Name:K.Sanjana

Roll.no:2303a52139

Batch.no:42

TASK1: Password Strength & Validation Check

Prompt: #write a python program for user registration script for weakpassword handling.valing password length, don't allow weak passwords.

Code:

```python
#write a python program for user registration script for weakpassword handling.valing password length,dont allow weak passwords.
def is_strong_password(password):

    if not isinstance(password, str):
        return False, "Password must be a string."

    if len(password) < 8:
        return False, "Password must be at least 8 characters long."

    has_upper = any(c.isupper() for c in password)
    has_lower = any(c.islower() for c in password)
    has_digit = any(c.isdigit() for c in password)
    has_special = any(not c.isalnum() for c in password)

    if not (has_upper and has_lower and has_digit and has_special):
        return False, "Password must include uppercase, lowercase, digit, and special character."

    return True, "Password is strong."
def register_user(username, password):

    if not isinstance(username, str) or not username.strip():
        return False, "Username must be a non-empty string."

    is_strong, message = is_strong_password(password)
    if not is_strong:
        return False, f"Password is weak: {message}"

    # Here you would normally save the user to a database
    return True, f"User '{username}' registered successfully."
# Example usage
if __name__ == "__main__":
    username = input("Enter username: ")
    password = input("Enter password: ")
    success, message = register_user(username, password)
    print(message)
```

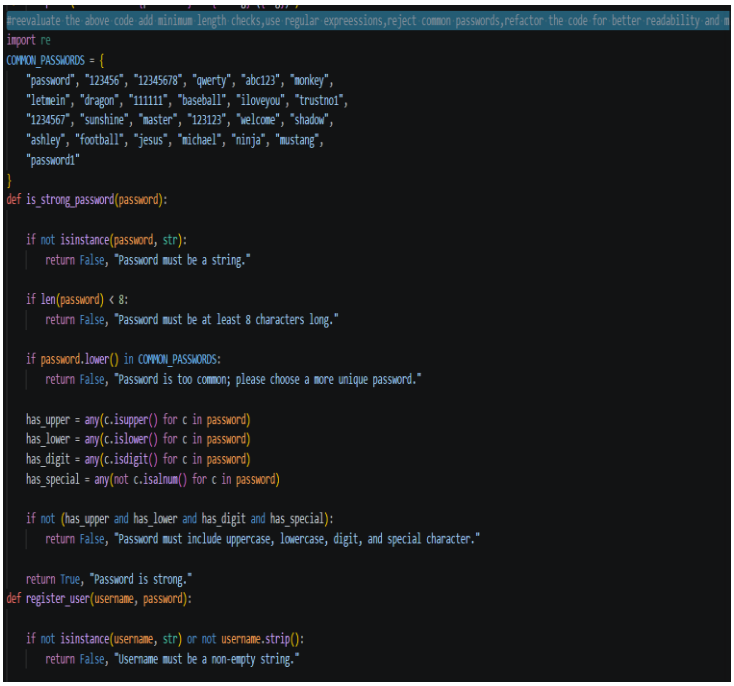
```python
# Run demo cases for password strength
demo_passwords = [
    "password",
    "Password1",
    "Pass1!",
    "P@ssw0rd",
    "12345678",
    "abcdefgh",
    "ABCDEFGH",
    "Abcdefg1",
]
print("\nDemo password strength results:")
for pwd in demo_passwords:
    strong, msg = is_strong_password(pwd)
    print(f"  Password={pwd!r:>15} -> {strong} ({msg})")
```

OUTPUT:

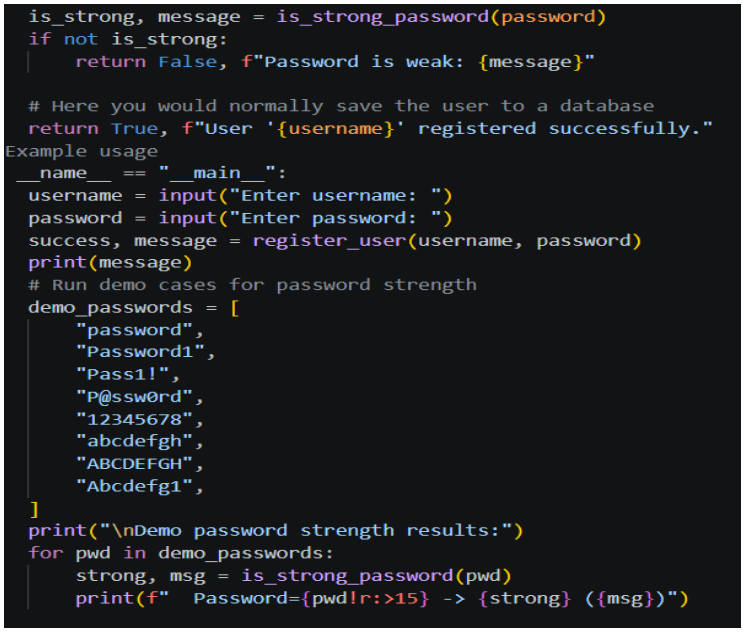
```
Enter username: sanjana kolipaka
Enter password: sanjana
Password is weak: Password must be at least 8 characters long.

Demo password strength results:
  Password=     'password' -> False (Password must include uppercase, lowercase, digit, and special character.)
  Password=    'Password1' -> False (Password must include uppercase, lowercase, digit, and special character.)
  Password=       'Pass1!' -> False (Password must be at least 8 characters long.)
  Password=     'P@ssw0rd' -> True (Password is strong.)
  Password=     '12345678' -> False (Password must include uppercase, lowercase, digit, and special character.)
  Password=     'abcdefgh' -> False (Password must include uppercase, lowercase, digit, and special character.)
  Password=     'ABCDEFGH' -> False (Password must include uppercase, lowercase, digit, and special character.)
  Password=     'Abcdefg1' -> False (Password must include uppercase, lowercase, digit, and special character.)
```

Modified: #reevaluate the above code add minimum length checks,use regular expreessions,reject common passwords,refactor the code for better readability and maintainability**.**

Code:



```python
#reevaluate the above code add minimum length checks,use regular expreessions,reject common passwords,refactor the code for better readability and m
import re
COMMON_PASSWORDS = {
    "password", "123456", "12345678", "qwerty", "abc123", "monkey",
    "letmein", "dragon", "111111", "baseball", "iloveyou", "trustno1",
    "1234567", "sunshine", "master", "123123", "welcome", "shadow",
    "ashley", "football", "jesus", "michael", "ninja", "mustang",
    "password1"
}
def is_strong_password(password):

    if not isinstance(password, str):
        return False, "Password must be a string."

    if len(password) < 8:
        return False, "Password must be at least 8 characters long."

    if password.lower() in COMMON_PASSWORDS:
        return False, "Password is too common; please choose a more unique password."

    has_upper = any(c.isupper() for c in password)
    has_lower = any(c.islower() for c in password)
    has_digit = any(c.isdigit() for c in password)
    has_special = any(not c.isalnum() for c in password)

    if not (has_upper and has_lower and has_digit and has_special):
        return False, "Password must include uppercase, lowercase, digit, and special character."

    return True, "Password is strong."
def register_user(username, password):

    if not isinstance(username, str) or not username.strip():
        return False, "Username must be a non-empty string."
```



```python
    is_strong, message = is_strong_password(password)
    if not is_strong:
        return False, f"Password is weak: {message}"

    # Here you would normally save the user to a database
    return True, f"User '{username}' registered successfully."
Example usage
__name__ == "__main__":
    username = input("Enter username: ")
    password = input("Enter password: ")
    success, message = register_user(username, password)
    print(message)
    # Run demo cases for password strength
    demo_passwords = [
        "password",
        "Password1",
        "Pass1!",
        "P@ssw0rd",
        "12345678",
        "abcdefgh",
        "ABCDEFGH",
        "Abcdefg1",
    ]
    print("\nDemo password strength results:")
    for pwd in demo_passwords:
        strong, msg = is_strong_password(pwd)
        print(f"  Password={pwd!r:>15} -> {strong} ({msg})")
```

Output:

```
Enter username: sanjana
Enter password: sanjana@1205
Password is weak: Password must include uppercase, lowercase, digit, and special character.

Demo password strength results:
  Password=     'password' -> False (Password is too common; please choose a more unique password.)
  Password=    'Password1' -> False (Password is too common; please choose a more unique password.)
  Password=       'Pass1!' -> False (Password must be at least 8 characters long.)
  Password=     'P@ssw0rd' -> True (Password is strong.)
  Password=     '12345678' -> False (Password is too common; please choose a more unique password.)
  Password=     'abcdefgh' -> False (Password must include uppercase, lowercase, digit, and special character.)
  Password=     'ABCDEFGH' -> False (Password must include uppercase, lowercase, digit, and special character.)
  Password=     'Abcdefg1' -> False (Password must include uppercase, lowercase, digit, and special character.)
```

Justification:

Security Goal: Reduce account compromise risk by enforcing length/complexity and blocking common passwords.

Threats Mitigated: Lowers success of brute-force, credential-stuffing, and trivial-guess attacks.

Design Choice: Server-side validation in `register_user` (file ass20.3.py) and clear, deterministic checks in `is_strong_password` for actionable feedback.

Operational Note: Never store plaintext—use salted hashing (bcrypt/Argon2); add rate-limiting and MFA for stronger protection.

Task 2 – Insecure File Upload Handling

Prompt: #generate a python file upload script that identify vulnerabilities such as:no file type validation,unrestricted file size.

Code:2. The user list is hardcoded, which is not scalable or secure for real applications.

```python
#generate a python file upload script that identify vulnerabilities such as:no file type validation,unrestricted file size.
import os
def validate_file(file_path):
    """Validate the uploaded file for type and size."""
    if not os.path.isfile(file_path):
        return False, "File does not exist."

    # Check file size (e.g., limit to 5 MB)
    max_size = 5 * 1024 * 1024  # 5 MB
    file_size = os.path.getsize(file_path)
    if file_size > max_size:
        return False, f"File size exceeds the limit of {max_size} bytes."

    # Check file type (e.g., allow only .txt and .pdf)
    allowed_extensions = {'.txt', '.pdf'}
    _, ext = os.path.splitext(file_path)
    if ext.lower() not in allowed_extensions:
        return False, f"File type '{ext}' is not allowed. Allowed types: {allowed_extensions}."

    return True, "File is valid."
def upload_file(file_path):
    """Simulate file upload and validate it."""
    is_valid, message = validate_file(file_path)
    if not is_valid:
        print(f"File upload failed: {message}")
        return False

    # Simulate file upload (e.g., move to a designated directory)
    # For demonstration, we just print a success message.
    print(f"File '{file_path}' uploaded successfully.")
    return True
```

```python
if __name__ == "__main__":
    import sys

    if len(sys.argv) > 1:
        fp = sys.argv[1]
    else:
        fp = input("Enter file path to upload: ").strip()

    success = upload_file(fp)
    if success:
        print("Result: upload succeeded.")
    else:
        print("Result: upload failed.")
```

Secure Code:

Output:

```
Enter file path to upload: D:\AI assistance coding\Task 1 – Password Strength & Valida.txt
File 'D:\AI assistance coding\Task 1 – Password Strength & Valida.txt' uploaded successfully.
Result: upload succeeded.
PS D:\AI assistance coding>
```

Modified prompt: #reevaluate the code and add file extension validation,sizelimits and file storage paths ,display messages accordingly,improve the overallcode

Code:

```python
import os
def validate_file(file_path):
    """Validate the file for existence, size, and type."""
    if not os.path.isfile(file_path):
        return False, "File does not exist."

    max_size = 5 * 1024 * 1024  # 5 MB
    file_size = os.path.getsize(file_path)
    if file_size > max_size:
        return False, f"File size exceeds the limit of {max_size} bytes."

    allowed_extensions = {'.txt', '.pdf'}
    _, ext = os.path.splitext(file_path)
    if ext.lower() not in allowed_extensions:
        return False, f"File type '{ext}' is not allowed. Allowed types: {allowed_extensions}."

    return True, "File is valid."
def upload_file(file_path):
    """Handle the file upload process."""
    is_valid, message = validate_file(file_path)
    if not is_valid:
        print(f"File upload failed: {message}")
        return False
```

```python
# Simulate file upload by moving it to a designated directory
upload_dir = "uploaded_files"
os.makedirs(upload_dir, exist_ok=True)
destination = os.path.join(upload_dir, os.path.basename(file_path))

try:
    os.rename(file_path, destination)
    print(f"File '{file_path}' uploaded successfully to '{destination}'.")
    return True
except Exception as e:
    print(f"Error during file upload: {e}")
    return False
__name__ == "__main__":
import sys

if len(sys.argv) > 1:
    fp = sys.argv[1]
else:
    fp = input("Enter file path to upload: ").strip()

success = upload_file(fp)
if success:
    print("Result: upload succeeded.")
else:
```

Output:

```
Enter file path to upload: D:\AI assistance coding\Task 1 - Password Strength & Valida.txt
File 'D:\AI assistance coding\Task 1 - Password Strength & Valida.txt' uploaded successfully to 'uploaded_files\Task 1 - Password Strength & Valida.txt'.
Result: upload succeeded.
PS D:\AI assistance coding>
```

Justification:

Security Goal:Prevent common file-upload attacks by enforcing size limits, allowed extensions, and validating existence before processing.

Threats Mitigated:Reduces risk of DoS from huge uploads, server compromise via executable/malicious files, and path-traversal or overwrite attacks.

Design Choices: Server-side checks in `validate_file()` (size + extension) give deterministic rejection; storing uploads under a dedicated uploaded_files directory isolates content.

Additional Hardening (recommended):Verify MIME/type by content (magic bytes), sanitize/normalize filenames, use secure unique names, store outside webroot, set restrictive permissions, scan files for malware, and apply rate-limits.

Operational Note: Never trust client input; log failures for incident detection and ensure uploaded data is handled with the same confidentiality/integrity controls as other user data.

Task3: Real-Time Application: API Security Review

Prompt: *Generate a Flask API endpoint to fetch user data.*

Insecure Code

```
@app.route('/users', methods=['GET'])

def get_users():

    return {"users": ["admin", "guest"]}
```

Vulnerabilities:

1. The endpoint exposes user information without authentication, which can lead to unauthorized access.

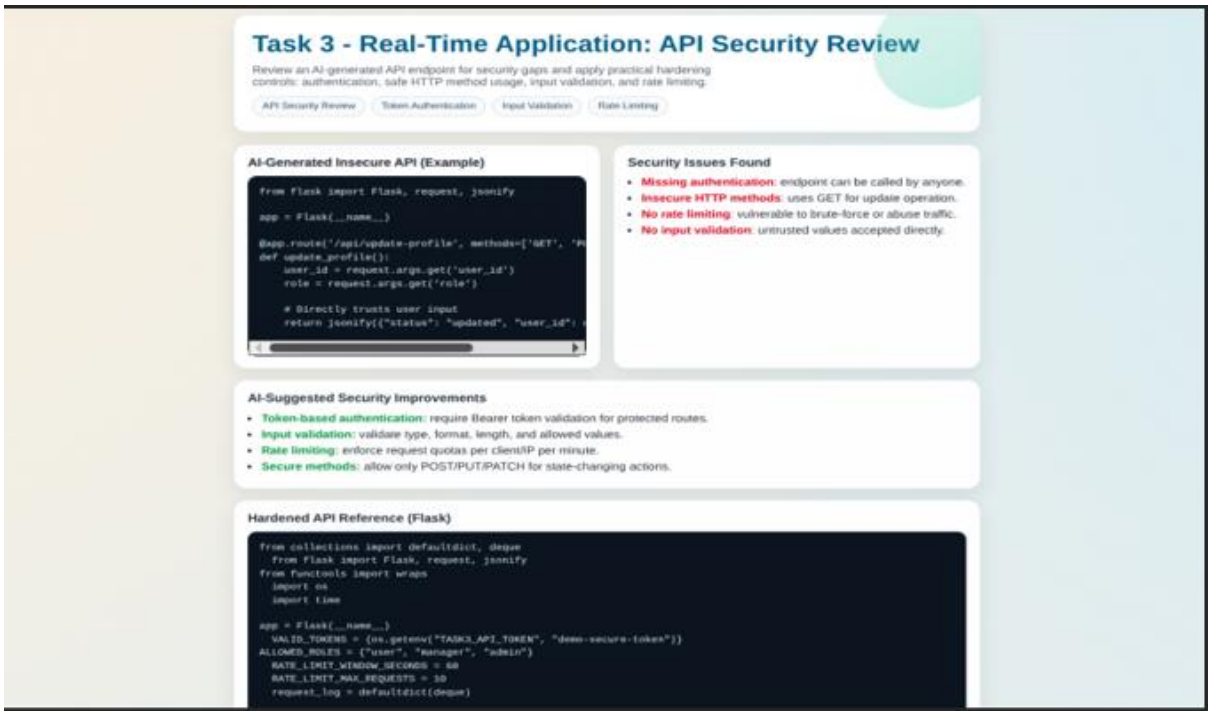
2. The user list is hardcoded, which is not scalable or secure for real applications.

Secure Code:

```
from flask import Flask, request, jsonify, abort

from functools import wraps



app = Flask(__name__)



VALID_TOKEN = "securetoken123"



def token_required(f):

    @wraps(f)

    def decorated():

        token = request.headers.get('Authorization')
```

```
        if token != VALID_TOKEN:
            abort(401, "Unauthorized")
        return f()
    return decorated


@app.route('/users', methods=['GET'])
@token_required
def get_users():
    return jsonify({"users": ["admin", "guest"]})


if __name__ == "__main__":
    app.run()
```

Output:



Enhancements Suggested

- Use JWT tokens instead of static tokens
- Implement rate limiting (Flask-Limiter)
- Add input sanitization

Justification

- Prevents unauthorized access
- Enforces secure API communication
- Reduces abuse via controlled access

Task 4: Cross-Site Scripting (XSS) Check

Prompt:

Generate an HTML feedback form with JavaScript output.

Insecure Code:

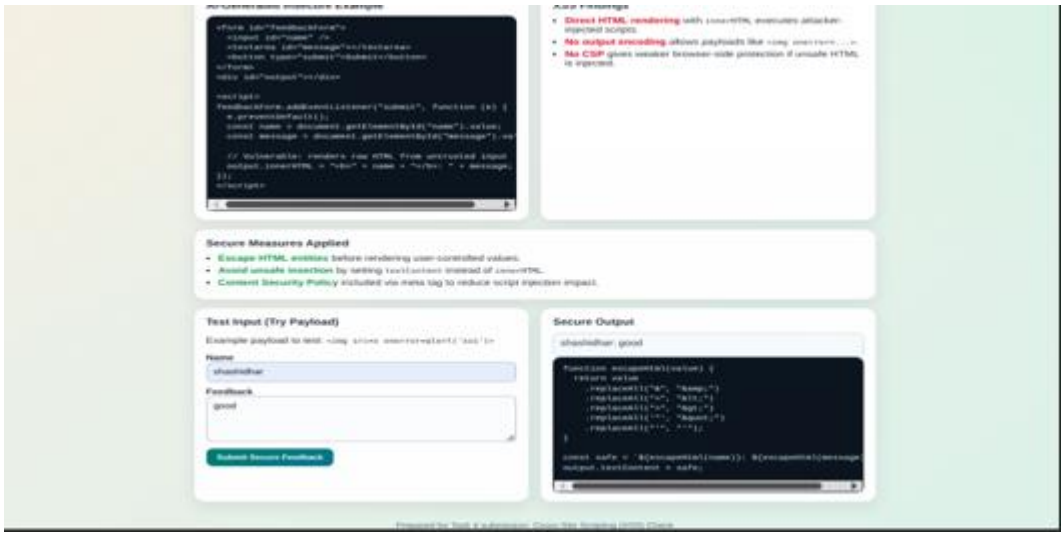
```
<input type="text" id="name">

<button onclick="submitForm()">Submit</button>



<p id="output"></p>



<script>

function submitForm() {

   let name = document.getElementById("name").value;
```

Secure Code:

```
<input type="text" id="name">

<button onclick="submitForm()">Submit</button>



<p id="output"></p>



<script>
```

```
function submitForm() {
   let name = document.getElementById("name").value;
   document.getElementById("output").innerHTML = name;
}
</script>
<script>alert('Hacked')</script>
<input type="text" id="name">
<button onclick="submitForm()">Submit</button>

<p id="output"></p>

<script>
function escapeHTML(str) {
   return str.replace(/[&<>"']/g, function(m) {
       return {
           '&': '&',
           '<': '<',
           '>': '>',
           '"': '"',
           "'": '''
       }[m];
   });
}
function submitForm() {
   let name = document.getElementById("name").value;
   document.getElementById("output").innerText = escapeHTML(name);
}
</script>
```

Output:



Explanation:

Prevents script injection

Uses output encoding instead of raw rendering

Eliminates DOM-based XSS

Task 5: SQL Injection Prevention

Prompt: *Generate a Python script to fetch user details from SQLite database.*

Code:

```
import sqlite3


conn = sqlite3.connect("users.db")

cursor = conn.cursor()


username = input("Enter username: ")
```

```
query = f"SELECT * FROM users WHERE username = '{username}'"

cursor.execute(query)



print(cursor.fetchall())
```

Vulnerabilities:

The code is vulnerable to SQL injection attacks, as it directly incorporates user input into the SQL query without sanitization or parameterization.

An attacker could manipulate the input to execute arbitrary SQL commands, potentially compromising the database.

Secure Code:

```
import sqlite3


def setup_db(connection):
    cursor = connection.cursor()
    cursor.execute("DROP TABLE IF EXISTS users")
    cursor.execute(
        """
        CREATE TABLE users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL,
            email TEXT NOT NULL
        )
        """
    )
    cursor.executemany(
        "INSERT INTO users (username, email) VALUES (?, ?)",
        [
            ("alice", "alice@example.com"),
            ("bob", "bob@example.com"),
            ("charlie", "charlie@example.com"),
        ],
    )
    connection.commit()
```

```python
def insecure_lookup(cursor, user_input):
    query = f"SELECT * FROM users WHERE username = '{user_input}'"
    cursor.execute(query)
    return cursor.fetchall()


def secure_lookup(cursor, user_input):
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (user_input,))
    return cursor.fetchall()


conn = sqlite3.connect(":memory:")
setup_db(conn)
cur = conn.cursor()

normal_input = "alice"
print("=== Normal lookup (alice) ===")
print("Insecure :", insecure_lookup(cur, normal_input))
print("Secure   :", secure_lookup(cur, normal_input))

payload = "' OR '1'='1"
print("\n=== Injection test payload ===")
print("Payload:", payload)
print("Insecure:", insecure_lookup(cur, payload))
print("Secure  :", secure_lookup(cur, payload))

conn.close()
```

Output:

Explanation:

Uses parameterized queries

Prevents SQL injection attacks

Ensures safe database interaction